

B

Argus Financial Partners

End-to-End UX & System Test Guide

ARGUS FINANCIAL PARTNERS · CONFIDENTIAL — FOUNDER · JUNE 27, 2026

Strategic guidance only — not legal, financial, tax, or investment advice. Securities, money-transmission, and corporate-formation decisions must be reviewed by licensed counsel and a CPA before you act. Sections flagged “see counsel” are where this matters most.

Contents

1. 0. The stack — what runs where

2. 1. First-time setup & test data

1.1 .env flags that unlock journeys

1.2 Test credentials & data created by seed:e2e

5. 2. Customer-portal journeys (start at <http://localhost:5173>)

J1 — Auth (passkey-first) & registration

J2 — Agentic onboarding & the tier ladder

J3 — Dashboard & money transfer (now on Temporal-capable path)

J4 — Invest (tokenized RWA / securities)

J5 — Collect (collectibles)

J6 — Agent / SmartChat (NL → 90s operation token → MFA gate)

J6b — Console (terminal-style agent; /console, Tier 2)

Portal money-app pages & the More menu

J7 — Trading (Phase 17 simulated broker; needs TRADING_ENABLED=true)

J8 — Escrow (chargeback substitute)

J9 — Argus Pay (merchant rail; needs ARGUS_PAY_ENABLED=true)

J9b — Bank rails: deposit, withdraw, statement (needs BANK_RAILS_ENABLED=true)

J9c — Debit cards: issue, authorize, capture/void/refund (needs CARDS_ENABLED=true)

J9d — Bill pay: payees + scheduled/recurring payments (needs BILLPAY_ENABLED=true)

20. 2a. Money in/out & wealth journeys (new cluster — J14–J23)

J14 — Add cash (fiat → USDC on-ramp) — /add-cash (needs ONRAMP_ENABLED=true)

J15 — Cash out (USDC → fiat off-ramp) — /cash-out (needs OFFRAMP_ENABLED=true)

J16 — Earn (tokenized Treasury, F1) — /earn (needs TREASURY_ENABLED=true)

J17 — Borrow (collateralized lending) — /borrow (needs LENDING_ENABLED=true + Treasury)

J18 — Requests (P2P money requests, F3) — /requests

J19 — Send abroad (cross-border, F6) — /send-abroad (needs FX_ENABLED +
FX_SETTLEMENT_ENABLED)

J20 — Currency exchange (FX) — /fx (needs FX_ENABLED=true)

J21 — Self-custody & portability (F2) — /self-custody

J22 — Drops (creator drops, F5) — /drops (needs CREATOR_DROPS_ENABLED=true)

J23 — Pay with Argus + login-less VC checkout — /pay (needs ARGUS_PAY_ENABLED /
CHECKOUT_VP_ENABLED)

J10 — On-chain wallet (only with HEDERA_ENABLED=true)

J11 — Credentials (DID / Verifiable Credentials)

J12 — Connected agents & grants (start at <http://localhost:5174>)

J13 — Admin console (RBAC; start at <http://localhost:5173/admin/login>)

35. 3. Back-office agent operations (Phase 15) — API + admin

36. 4. The fraud system — how it works, logs, and learns

How to test the fraud journey

38. 5. Backend end-to-end testing (build · start · stop · validate)

39. 6. Optional: durable orchestration (Temporal + Conductor)

40. 7. Journey × channel coverage matrix

41. 8. Where to add agents, skills & MCP services (expansion review)

42. 9. Troubleshooting

Argus Financial Partners — End-to-End UX & System Test Guide

A hands-on guide to exercising **every product, every channel, and the fraud platform** as a real user/operator, plus how to build/run/validate the backend end-to-end. This is the manual, "drive the actual apps" companion to the automated runbook in `docs/E2E-VALIDATION.md` (deterministic vitest + the `e2e-validator` skill).

- Use **this doc** to click through journeys, see the fraud system react, and demo the product.
- Use `E2E-VALIDATION.md` for the repeatable pass/fail gate (CI-style).

1 0. The stack — what runs where

#	Service	Start in	URL / port	Needed for
1	Backend API	<code>backend/</code>	<code>http://localhost:3001</code> (health: <code>/api/health</code>)	everything
2	Customer portal (React)	<code>frontend/</code>	<code>http://localhost:5173</code>	all user journeys
3	External agent app (OID4VP + MCP)	<code>argus-agent/</code>	<code>http://localhost:5174</code>	connected-agent journey
4	Fraud engine (standalone)	<code>fraud-engine/</code>	<code>http://localhost:4500</code> (health: <code>/health</code>)	fraud journey, learning loop
5	Temporal (optional)	<code>backend/</code> compose	gRPC <code>localhost:7233</code> , UI <code>http://localhost:8233</code>	durable money + agent ops
6	Conductor (optional)	<code>backend/</code> compose	API <code>http://localhost:8080/api</code> , UI <code>http://localhost:5001</code>	agent-ops orchestration
7	iOS wallet (<code>ArgusWallet/</code>)	Xcode	simulator	mobile (unverified source — needs macOS/Xcode)

Primary starting URL for almost everything: `http://localhost:5173` (the customer portal). Admin journeys start at `http://localhost:5173/admin/login`. The external-agent journey starts at `http://localhost:5174`.

2 1. First-time setup & test data

```
# --- backend ---
cd backend
npm install
cp .env.example .env          # then edit (see §1.1)
npm run seed:e2e              # migrate + seed RBAC admin + register simulator MCP c
                              # + 5 demo users + marketplace assets (idempotent)
npm run dev                   # API on :3001 (auto-migrates in dev)

# --- customer portal (new terminal) ---
cd frontend && npm install && npm run dev      # :5173

# --- external agent app (new terminal) ---
cd argus-agent && npm install && npm run dev    # :5174

# --- fraud engine (new terminal) ---
cd fraud-engine && npm install && cp .env.example .env && npm run dev  # :4500
```

The money in/out & wealth cluster (J14–J23: Add cash, Cash out, Earn, Borrow, FX, Send abroad, Drops, login-less checkout) adds **no new service** — it's all in the backend + portal. It only needs the §1.1 flags set (the dev `.env` already ships several on).

1.1 `.env` flags that unlock journeys

Most products are behind kill-switches (off by default). For a full demo set these in `backend/.env`:

```

ALLOW_PASSWORD_AUTH=true          # lets you log in with the demo passwords (passkeys
TRADING_ENABLED=true              # /trade journey (Phase 17 simulated broker)
ARGUS_PAY_ENABLED=true            # Argus Pay merchant journey (Phase 21)
BANK_RAILS_ENABLED=true          # bank rails: deposit/withdraw/statement (Phase 19 S
CARDS_ENABLED=true                # debit cards: issue/authorize/capture (Phase 19.4)
BILLPAY_ENABLED=true             # bill pay: payees + scheduled/recurring payments (P
# Money in/out & wealth cluster (J14-J23):
ONRAMP_ENABLED=true              # Add cash – fiat→USDC on-ramp (/add-cash)
OFFRAMP_ENABLED=true             # Cash out – USDC→fiat off-ramp (/cash-out)
TREASURY_ENABLED=true            # Earn – tokenized Treasury (/earn) – also the Borro
LENDING_ENABLED=true             # Borrow – collateralized lending (/borrow); needs T
FX_ENABLED=true                  # Currency exchange (/fx)
FX_SETTLEMENT_ENABLED=true       # Send abroad – cross-currency settlement (/send-abr
CREATOR_DROPS_ENABLED=true       # Drops – creator drops (/drops)
CHECKOUT_VP_ENABLED=true        # login-less VC checkout on /pay
JOURNEYS_ENABLED=true            # journey-orchestration platform (/api/journeys)
# CARD_CASHBACK_BPS=150          # optional: card cashback % (USDC) paid on capture
# Fraud engine wiring (both sides must share the key):
FRAUD_ENGINE_URL=http://localhost:4500
FRAUD_ENGINE_API_KEY=<a-strong-shared-secret-32+chars>
# Optional orchestration (see §9):
# TEMPORAL_MONEY_ENABLED=true    CONDUCTOR_ENABLED=true    TEMPORAL_ENABLED=true
# Optional on-chain (else simulated):
# HEDERA_ENABLED=true  HEDERA_OPERATOR_ID=...  HEDERA_OPERATOR_KEY=...  HEDERA_USDC_TOKEN_

```

In `fraud-engine/.env` set the **same** `FRAUD_ENGINE_API_KEY` (the engine validates it and reuses it for freeze callbacks).

⚠ These flags are **prod-fatal** — the server refuses to boot in `NODE_ENV=production` with `TRADING_ENABLED` / `ARGUS_PAY_ENABLED` / `ALLOW_PASSWORD_AUTH` on, with the simulated on/off-ramp (`ONRAMP_ENABLED` / `OFFRAMP_ENABLED` while `*_PROVIDER=simulated`), with `LENDING_ENABLED` or the simulated money/wealth seams, or with `KMS_PROVIDER=local` . They are for local/dev demoing only. The dev `.env` already ships the on-ramp, off-ramp, Treasury, and lending flags **on**.

1.2 Test credentials & data created by `seed:e2e`

Who	Login	Tier / state	Use for
<code>admin@argusfinancial.com</code> / <code>Admin1234!</code>	<code>/admin/login</code>	RBAC <code>admin</code>	admin console, agent-ops, escrow mediation
<code>alex@demo.com</code> / <code>Demo1234!</code>	<code>/login</code>	Tier 2, \$12,500	transfers, SmartChat, escrow
<code>blair@demo.com</code> / <code>Demo1234!</code>	<code>/login</code>	Tier 2, \$40,000	marketplace Invest (accredited)
<code>casey@demo.com</code> / <code>Demo1234!</code>	<code>/login</code>	Tier 1, \$3,000	KYC step-up; rejection demo (expired doc #1)
<code>drew@demo.com</code> / <code>Demo1234!</code>	<code>/login</code>	Tier 0, \$750	fresh signup; rejection demo (tampered doc #2)
<code>erin@demo.com</code> / <code>Demo1234!</code>	<code>/login</code>	Tier 0, \$250	fresh signup; rejection demo (low-quality doc #3)

Plus: marketplace Invest/Collect assets, and the simulator MCP client

`did:simulator:agent-app` (used by the external-agent app, \$500 ceiling).

Extra data created inline per journey below (a merchant for Argus Pay, an agent-ops review) via `curl`.

3 2. Customer-portal journeys (start at `http://localhost:5173`)

Each journey lists the **start URL**, steps, and **what to verify**. Log in as the noted demo user.

J1 — Auth (passkey-first) & registration

- **Start:** `/register` → `/login`.
- Register a new user (passkey if your browser supports WebAuthn; password form appears only when `ALLOW_PASSWORD_AUTH=true`). Then log in.
- **Verify:** session persists; `Idempotency-Key` is auto-attached to money POSTs (Network tab); 5 failed password attempts → `ACCOUNT_LOCKED` (reset with `npm run reset:auth`).

J2 — Agentic onboarding & the tier ladder

- **Start:** `/onboarding` (log in as **drew** or **erin**, Tier 0).

- Submit signals/document. The risk orchestrator scores signals → may require step-up (document/possession). Low confidence or a "weak" doc number routes to **manual review**.
- **Verify:** the tier-ladder dots advance; a clean flow grants Tier 2; **drew/erin/casey** (seeded with rejection doc numbers 1/2/3) land in the admin review queue (see J13).

J3 — Dashboard & money transfer (now on Temporal-capable path)

- **Start:** `/` as alex.
- Send a transfer to **blair** (by email). Amounts render only from integer minor units.
- **Verify:** balance updates; `/activity` shows `transfer_out` + matching `transfer_in`; a **replay with the same Idempotency-Key posts no second journal** (exactly-once). With `TEMPORAL_MONEY_ENABLED` the transfer runs as a durable Temporal workflow (see the worker logs / Temporal UI :8233) — same result, because the ledger is the source of truth.

J4 — Invest (tokenized RWA / securities)

- **Start:** `/invest` as **blair** (accredited, Tier 2).
- Open an asset (`/asset/:id`) → **quote** (see fee/source/as-of/staleness) → **TradeSheet** confirm.
- **Verify:** a single atomic cash+asset+fee journal; holdings appear; a Tier-1 user (**casey**) is compliance-gated (`COMPLIANCE_BLOCKED` / `TIER_REQUIRED`).

J5 — Collect (collectibles)

- **Start:** `/collect` . Buy/sell a collectible; same quote→confirm flow.
- **Verify:** holding reflects in the ledger; secondary sell posts the atomic journal.

J6 — Agent / SmartChat (NL → 90s operation token → MFA gate)

- **Start:** `/agent` as **alex** (Tier 2 required).
- Type "send \$50 to `blair@demo.com`". Watch the **90-second token countdown**. Try ">\$500" to trigger the **MFA gate**.
- **Verify:** intent classified → scoped 90s RS256 operation token → transfer posts idempotently on the token id; an expired token is rejected; MFA required above \$500.
- **SmartChat now also drives the money rails** (Phase 19): try "deposit \$50", "withdraw \$20 to my bank", and `pay "City Power" $90` (add the biller on the Bills page first). Deposits are money-in → never MFA-gated; a >\$500 withdrawal/bill prompts for MFA.

J6b — Console (terminal-style agent; `/console`, Tier 2)

- **Start:** `/console`. Type `help`, then commands like `balance`, `deposit $50`, `pay "City Power" $90`, `send $20 to blair@demo.com`. It routes each line through the SmartChat agent (same 90s token + MFA); an MFA-gated command flips the prompt to `mfa>` for the code.

Portal money-app pages & the More menu

The customer portal surfaces every product through a flat top nav (Home · Invest · Collect · Agent) plus a **grouped More menu** — **Money** (Add cash · Cash out · Earn · Borrow · Bank · Cards · Bills · Requests · Send abroad · Currency exchange · Argus Pay · Escrow), **Invest & collect** (Trade · Drops), **Trust & identity** (Self-custody · On-chain wallet · Credentials · Verification & tiers · Connected agents · Internal agents), **Family** (Starter guardian/teen), and **Account** (Activity · Console). On wide screens these appear in the sidebar. Each gated page needs its matching `*_ENABLED` flag (\$1.1).

J7 — Trading (Phase 17 simulated broker; needs `TRADING_ENABLED=true`)

- **Start:** `/trade` as alex.
- Place a simulated equity/crypto order.
- **Verify:** order accepted on the hot path; settlement posts into the ledger **asynchronously** and idempotently; trading is bulkheaded — a broker stall doesn't block J3 transfers (SLA isolation).

J8 — Escrow (chargeback substitute)

- **Start:** `/escrow` as alex (payer) → hold funds for blair.
- **Verify:** hold debits payer to the `escrow` system account; release→credits recipient; refund→back to payer; dispute → mediated by admin (J13). Each step is an idempotent ledger journal.

J9 — Argus Pay (merchant rail; needs `ARGUS_PAY_ENABLED=true`)

Create a merchant first (as blair, Tier 2). Get a session token by logging in via the portal and copying the Bearer from the Network tab, or use the API:

```
# register merchant (blair is the settlement account)
curl -sX POST localhost:3001/api/pay/merchants -H "Authorization: Bearer $BLAIR" \
  -H 'Content-Type: application/json' -d '{"name":"Acme Coffee"}'
# merchant requests money (idempotent)
curl -sX POST localhost:3001/api/pay/intents -H "Authorization: Bearer $BLAIR" \
  -H 'Idempotency-Key: pay-1' -H 'Content-Type: application/json' \
  -d '{"merchantId":"<id>","amountMinor":"500","currency":"USD"}'
```

- Pay the intent as **alex**, then capture as **blair**; try a refund and a payer dispute.
- **Verify:** every payment is **escrow-protected** (status derives from the escrow row); zero rail fee; agents can pay via the `pay_merchant` MCP tool (scope `pay:merchant`).

J9b — Bank rails: deposit, withdraw, statement (needs `BANK_RAILS_ENABLED=true`)

- **API** (Tier 2; get `$ALEX` Bearer from the portal Network tab). The simulated partner bank settles instantly into the ledger via `external_clearing`.

```
# on-ramp (deposit) – Idempotency-Key required
curl -sX POST localhost:3001/api/bank/deposit -H "Authorization: Bearer $ALEX" \
  -H 'Idempotency-Key: dep-1' -H 'Content-Type: application/json' -d '{"amountMinor":""
# off-ramp (ACH payout)
curl -sX POST localhost:3001/api/bank/withdraw -H "Authorization: Bearer $ALEX" \
  -H 'Idempotency-Key: wd-1' -H 'Content-Type: application/json' -d '{"amountMinor":""2
curl -s "localhost:3001/api/bank/statement?from=1970-01-01T00:00:00Z&to=2999-01-01T00:
# admin: ACH return reversal + FBO coverage (compliance/admin)
curl -sX POST localhost:3001/api/admin/bank/transfers/<id>/return -H "Authorization: B
curl -s localhost:3001/api/admin/bank/fbo?currency=USD -H "Authorization: Bearer $ADMI
```

- **Verify:** deposit credits / withdraw debits `user_cash` (visible in `/activity`); replaying an Idempotency-Key posts no second journal; an over-balance withdrawal → `INSUFFICIENT_FUNDS`; a frozen account (fraud, J\$4) → `ACCOUNT_FROZEN`; the statement's closing balance reconciles to the ledger; FBO coverage shows the partner bank backing customer cash 1:1; a return reverses the journal.

J9c — Debit cards: issue, authorize, capture/void/refund (needs

CARDS_ENABLED=true)

```
# issue a card (Tier 2)
curl -sX POST localhost:3001/api/cards -H "Authorization: Bearer $ALEX"

# simulate a purchase – places a hold on funds (Idempotency-Key required)
curl -sX POST localhost:3001/api/cards/<cardId>/authorize -H "Authorization: Bearer $A"
-H 'Idempotency-Key: auth-1' -H 'Content-Type: application/json' -d '{"amountMinor":
# cardholder cancels before capture
curl -sX POST localhost:3001/api/cards/authorizations/<authId>/void -H "Authorization:
# merchant/processor side (admin): capture or refund
curl -sX POST localhost:3001/api/admin/cards/authorizations/<authId>/capture -H "Autho
curl -sX POST localhost:3001/api/admin/cards/authorizations/<authId>/refund -H "Autho
```

- **Verify:** only a masked PAN (••••last4) is returned; authorize **holds** funds (spendable cash drops into `card_holds`); capture settles out via `external_clearing` (money gone); void releases the hold back; refund returns a captured amount; replaying the auth Idempotency-Key places no second hold; over-balance → `INSUFFICIENT_FUNDS` ; a frozen account → `ACCOUNT_FROZEN` .
- **Cashback (F4):** with `CARD_CASHBACK_BPS` set, capture pays USDC cashback from a `card_rewards` system account (idempotent), viewable on `/cards` (`GET /api/cards/rewards`) — spend on the native rail.

J9d — Bill pay: payees + scheduled/recurring payments (needs

BILLPAY_ENABLED=true)

```
# save a biller
curl -sX POST localhost:3001/api/billpay/payees -H "Authorization: Bearer $ALEX" \
-H 'Content-Type: application/json' -d '{"name":"City Power","category":"utility","l
# pay now (Idempotency-Key); or schedule with "scheduledFor":"2026-07-01T00:00:00Z" +
curl -sX POST localhost:3001/api/billpay/pay -H "Authorization: Bearer $ALEX" \
-H 'Idempotency-Key: bp-1' -H 'Content-Type: application/json' -d '{"payeeId":"<id>"
curl -sX POST localhost:3001/api/billpay/payments/<id>/cancel -H "Authorization: Beare
# ops due-loop (admin): settle all due scheduled payments
curl -sX POST localhost:3001/api/admin/billpay/process -H "Authorization: Bearer $ADMI
```

- **Verify:** pay-now debits `user_cash` via `external_clearing` (visible in `/activity`); replaying the Idempotency-Key pays once; over-balance → `INSUFFICIENT_FUNDS` ; frozen

→ `ACCOUNT_FROZEN`; a future-dated payment stays `scheduled` until the due-loop settles it; a recurring payment seeds its next instance on send.

4 2a. Money in/out & wealth journeys (new cluster — J14–J23)

Get money onto the rail, off it, grow it, and borrow against it — all on the own USDC-on-Hedera + double-entry-ledger rail. Each rides a balanced, idempotent ledger journal (money is integer minor units). Set the \$1.1 flags; they're all in the **More** → **Money** menu (and the wide sidebar). Log in as **alex** unless noted.

J14 — Add cash (fiat → USDC on-ramp) — `/add-cash` (needs `ONRAMP_ENABLED=true`)

- **Start:** `/add-cash`. Enter a USD amount → the **live quote** shows the on-ramp fee and the USDC you'll receive → **Buy USDC**.
- **Verify:** USDC lands in your balance; the journal is `onramp_settlement` → `user_cash` (net) + `fee`; replaying the Idempotency-Key credits once; a frozen account → `ACCOUNT_FROZEN`. The simulated provider delivers instantly; a real provider (MoonPay/Stripe/Coinbase) returns a hosted-widget redirect and credits on its webhook. API: `POST /api/onramp/quote`, `POST /api/onramp/order` (Idempotency-Key).

J15 — Cash out (USDC → fiat off-ramp) — `/cash-out` (needs `OFFRAMP_ENABLED=true`)

- **Start:** `/cash-out`. Enter a USDC amount → quote shows the fee and the fiat you'll receive → **Cash out** (optional linked destination).
- **Verify:** debits `user_cash(USDC)` → `offramp_settlement` (net) + `fee`; the **money-leaving guards** apply (account-freeze, fraud-screen, authoritative balance check) so an over-sell → `INSUFFICIENT_FUNDS` and a frozen account → `ACCOUNT_FROZEN`; idempotent on replay. Closes the loop with J14. API: `POST /api/offramp/quote`, `POST /api/offramp/order` (Idempotency-Key).

J16 — Earn (tokenized Treasury, F1) — `/earn` (needs `TREASURY_ENABLED=true`)

- **Start:** `/earn`. Buy ATB (\$1 par) with USD cash; see your position. Accrue the period's yield from the admin/ops side (`POST /api/admin/...` or the daily loop).
- **Verify:** subscribe/redeem post a balanced par cash↔token journal; **yield distributes pro-rata to holders** automatically (the anti-6%-APY — you *own* a yield-bearing asset, not a freezable balance).

J17 — Borrow (collateralized lending) — `/borrow` (needs `LENDING_ENABLED=true` + Treasury)

- **Start:** buy some ATB on `/earn` first, then `/borrow`. Set tokens to pledge → **live borrowing power** ($\leq 50\%$ LTV at par) → enter an amount → **Borrow**. Repay inline from the loan row.
- **Verify:** opening locks the asset (`user_asset` → `loan_collateral`) and disburses (`lending_pool` → `user_cash`); a borrow over the cap → `LTV_EXCEEDED` ; the health bar warns before liquidation; **full repayment releases the collateral** back to your holding. Admin/ops: `POST /api/admin/lending/loans/:id/accrue` advances interest, `POST /api/admin/lending/loans/:id/liquidate` seizes when debt breaches the 75% liquidation LTV (returns any surplus to the user).

J18 — Requests (P2P money requests, F3) — `/requests`

- **Start:** `/requests`. Request money from blair; switch to blair to pay (or decline) the received request; cancel one you sent.
- **Verify:** fulfilling rides the shared `executeTransfer` (idempotent key `p2p:req:<id>`) and posts a balanced `transfer_out` / `transfer_in` journal — visible in both users' `/activity`.

J19 — Send abroad (cross-border, F6) — `/send-abroad` (needs `FX_ENABLED` + `FX_SETTLEMENT_ENABLED`)

- **Start:** `/send-abroad`. Quote a corridor (e.g. USD → EURC) → send to another user in a different currency.
- **Verify:** one balanced cross-currency journal joined by the `fx_settlement` treasury account, with the FX spread booked explicitly as a `fee` — the recipient is credited net in the target currency.

J20 — Currency exchange (FX) — `/fx` (needs `FX_ENABLED=true`)

- **Start:** `/fx`. Convert between supported currencies (USD ↔ USDC/USDT/EURC); see the ppm rate + spread before confirming.
- **Verify:** the currency registry allowlist rejects an unsupported/disabled currency; a conversion posts a balanced two-currency journal (spread → `fee`). API: `GET /api/fx/currencies` , `POST /api/fx/quote` , `POST /api/fx/convert` .

J21 — Self-custody & portability (F2) — `/self-custody`

- **Start:** `/self-custody`. View the self-custody report, the **signed attestation**, and the export manifest.

- **Verify:** the report shows the server holds **no** wallet/Hedera private key (non-custodial); the attestation JWT is RS256-signed by the issuer and **verifies against the issuer JWKS** — proof you can walk away with your keys + credentials.

J22 — Drops (creator drops, F5) — `/drops` (needs `CREATOR_DROPS_ENABLED=true`)

- **Start:** `/drops` . Create a limited-edition drop (as a creator); **claim** an edition (as a buyer).
- **Verify:** scarcity is enforced at the ledger — `treasury → buyer` token + `buyer → creator` cash in one atomic journal; the claim is idempotent; the edition count can't be oversold.

J23 — Pay with Argus + login-less VC checkout — `/pay` (needs `ARGUS_PAY_ENABLED` / `CHECKOUT_VP_ENABLED`)

- **Start:** `/pay` . Register a merchant, create a payment intent, and **pay with a device Verifiable Credential — no login, no redirect to a card site** (the browser holds an ES256 `did:key` ; see `frontend/src/lib/deviceWallet.ts`).
- **Verify:** the device VC is presented and verified, then the payment posts **escrow-protected** (status derives from the escrow row — same model as J9). Ties together the Argus Pay rail (J9) and the OID4VP presentation path (J12) for a frictionless, self-sovereign checkout.

J10 — On-chain wallet (only with `HEDERA_ENABLED=true`)

- **Start:** `/wallet` . Provision a Hedera account → Receive (QR) → Send USDC.
- **Verify:** on-chain transfer posts a matching ledger journal; balances reconcile; settlement is gated by the daily reconciliation (`RECONCILIATION_HOLD`) and the signer custody mode (`HEDERA_SIGNER`). Without Hedera creds this journey is simulated/skipped.

J11 — Credentials (DID / Verifiable Credentials)

- **Start:** `/credentials` . View your issued VC; check JWKS at `GET /api/credentials/.well-known/...` .
- **Verify:** VC is RS256-signed by the issuer; revocation flips status (`BitstringStatusList`) and a revoked VC is rejected at the MCP gate (J12).

J12 — Connected agents & grants (start at `http://localhost:5174`)

- **Start:** the external agent app `http://localhost:5174` .
- Link your account (issues a VC + binds the wallet `did:key` + grant), then chat: the app does `detectIntent → present/challenge → wallet-signed VP → 90s scoped token → mcp/call` .

- In the portal, start at `/permissions` to see/revoke the grant.
- **Verify:** VP signature verified before access; `SCOPE_DENIED` when out of scope; a transfer posts a balanced journal; `REPLAY_DETECTED` on a reused nonce/VP; revoking the grant blocks further calls.

J13 — Admin console (RBAC; start at `http://localhost:5173/admin/login`)

- Log in as `admin@argusfinancial.com`.
- **Verify:** onboarding **review queue** (approve/reject drew/erin/casey), escrow **dispute mediation**, MCP-client **suspend**, **reconciliation** view, and the back-office **agent-ops** surface (§3). Sensitive actions require `compliance` / `admin` and are written to `audit_logs` with the actor id.

5 3. Back-office agent operations (Phase 15) — API + admin

The internal AI agents run on the operations runner. **Invariant:** agents recommend; a deterministic, RBAC-checked, audited gate executes; humans approve anything material. Drive them via the admin API (get `$ADMIN` Bearer by logging into `/admin/login` and copying the token, or via `/api/admin/...`).

```
ADMIN=<admin bearer>

# KYC review (drafts a recommendation, queues for compliance)
curl -sX POST localhost:3001/api/admin/agent-ops/kyc-review -H "Authorization: Bearer $ADMIN" \
  -H 'Content-Type: application/json' -d '{"userId":"<casey-id>","fullName":"Casey Mor"

# generic trigger for any registered skill:
curl -sX POST localhost:3001/api/admin/agent-ops/run -H "Authorization: Bearer $ADMIN" \
  -H 'Content-Type: application/json' -d '{"skill":"sanctions-rescreen","input":{"user"
curl -sX POST localhost:3001/api/admin/agent-ops/run -H "Authorization: Bearer $ADMIN" \
  -H 'Content-Type: application/json' -d '{"skill":"compliance-filing","input":{"filing"

# review queue + overdue (SLA breaches) + the append-only run trail:
curl -s localhost:3001/api/admin/agent-ops/reviews -H "Authorization: Bearer $ADMIN"
curl -s localhost:3001/api/admin/agent-ops/reviews/overdue -H "Authorization: Bearer $
curl -s localhost:3001/api/admin/agent-ops/runs/<workflowRun> -H "Authorization: Beare
# a human (compliance) resolves a queued review:
curl -sX POST localhost:3001/api/admin/agent-ops/reviews/<id>/decision -H "Authorizati
  -H 'Content-Type: application/json' -d '{"decision":"approve","reason":"verified"}'
```


Skills to exercise: `kyc-review` (human-gated tier grant), `sanctions-rescreen` (clean auto-passes; "Blocked Person" → escalate + 10-day OFAC deadline + human-approved freeze), `compliance-filing` (SAR 30d / OFAC 10d / CTR 15d, agent-drafted/human-filed), `support-response`, `incident-summary` (SRE), `marketing-draft` (auto < 1,000 recipients, admin gate ≥ 1,000), `marketplace-dd`.

Verify: a queued item appears in the review queue with a `due_at` for compliance items; only `compliance` / `admin` can resolve sanctions/filing items (support is `FORBIDDEN`); approving a KYC review grants Tier 2 via the deterministic `completeKycDecision` (the agent never grants); `agent_runs` is append-only. With `CONDUCTOR_ENABLED` / `TEMPORAL_ENABLED` the same flow runs through that orchestrator (see §9) and **degrades to in-process** if the server is down.

6 4. The fraud system — how it works, logs, and learns

Architecture (hybrid): the backend's in-app triage (`fraudService`) classifies each money event and routes it:

- **Blocking** (`fraudClient.scoreSync` → `POST :4500/v1/events?mode=score`) — waits for an advisory score, merges it, then applies a **local deterministic gate** (the engine is advisory; Argus decides). Degrades **open** unless `FRAUD_REMOTE_REQUIRED=true`.
- **Fire-and-forget** (`emitAsync` → `POST :4500/v1/events?mode=async`) — no wait. On a **severe** async decision the engine calls back `POST :3001/api/internal/remediation/freeze` (service-bearer auth) to **freeze** the account → append-only `account_holds` → derived `isAccountFrozen` → new transfers/pays hit the `ACCOUNT_FROZEN` gate. Idempotent on `decisionId`.

Inside the engine (`:4500`): event bus + schema registry → per-user **feature store** + enrichment → ensemble (`rules-v1`) + sequence model (`seq-v0`) via a model **registry** + **serving** with **config-driven routing** + **shadow/canary** → append-only **decision** topic → analyst **case queue** → async **remediation** → **label** → **retrain** loop that registers shadow candidates.

How to test the fraud journey

1. Wire the engine (§1.1) and start it (`:4500`). Confirm `curl localhost:4500/health`.
2. Trigger scored events by doing **transfers** in the portal (J3) / SmartChat (J6) — `fraudService` screens new, fundable transfers.

3. Inspect what it logged (service-bearer = `FRAUD_ENGINE_API_KEY`):

```
FK=<FRAUD_ENGINE_API_KEY>
curl -s localhost:4500/v1/decisions -H "Authorization: Bearer $FK"      # recent
curl -s localhost:4500/v1/cases -H "Authorization: Bearer $FK"        # analyst
curl -s localhost:4500/v1/cases/<id> -H "Authorization: Bearer $FK"
```

Backend side: the append-only `fraud_decisions` table + `fraud_decision_total` / `fraud_remote_call_total` / `account_hold_total` metrics at `localhost:3001/metrics`.

4. See it freeze: push a severe async event (large/structured velocity) → engine calls back → the payer's next transfer returns `ACCOUNT_FROZEN`. Inspect `account_holds`; unfreeze via the engine's `/v1/cases/:id/action` or `POST :3001/api/internal/remediation/unfreeze`.

5. See it learn: label a decision and retrain:

```
curl -sX POST localhost:4500/v1/labels -H "Authorization: Bearer $FK" \
  -H 'Content-Type: application/json' -d '{"decisionId":"<id>","label":"fraud"}'
curl -sX POST localhost:4500/v1/retrain -H "Authorization: Bearer $FK"      # regis
curl -s localhost:4500/v1/models -H "Authorization: Bearer $FK"            # see v
curl -s localhost:4500/v1/routing -H "Authorization: Bearer $FK"          # curre
curl -sX PUT localhost:4500/v1/routing -H "Authorization: Bearer $FK" \
  -H 'Content-Type: application/json' -d '{"primary":"rules-v1","shadow":"seq-v0",'
# promote a candidate once it earns trust:
curl -sX POST localhost:4500/v1/models/<version>/promote -H "Authorization: Bearer
```

Verify the learning loop: label → retrain registers a new shadow model → routing can canary it → promotion makes it primary. The decision topic, cases, and `fraud_decisions` are append-only (audit).

The real Kafka/Flink/Triton/MLflow/lakehouse are the production swap; the interfaces map 1:1. See `docs/business/FraudEngine-GapAnalysis.md`.

7 5. Backend end-to-end testing (build · start · stop · validate)

```
cd backend

# Build / typecheck
npm run typecheck          # tsc --noEmit (run after every change)
npm run build              # compile to dist/ (+ copies SQL migrations)

# Start / stop
npm run dev                # dev server :3001 (auto-migrates); Ctrl-C to stop
npm start                  # run compiled dist/ (after build)
npm run migrate            # apply migrations explicitly

# Validate – deterministic suite (the money/security invariants + per-phase suites)
npm test                   # vitest run (full suite – currently 381 pass / 3 todo)
npx vitest run e2e         # just the e2e journey suite
npx vitest run test/invariants.test.ts      # money no-float, balanced journals, ide
npx vitest run test/kms.test.ts test/signer.test.ts  # custody / HSM-signer seam
npx vitest run test/operations.test.ts test/compliance.test.ts test/backoffice-skills.
# money in/out & wealth cluster (J14–J23):
npx vitest run test/onramp.test.ts test/offramp.test.ts test/lending.test.ts test/trea
npx vitest run test/fx.test.ts test/cross-border.test.ts test/payment-requests.test.ts
  test/creator-drops.test.ts test/self-custody.test.ts test/checkout-vp.test.ts test/j

# Validate – live orchestration (need the matching server up; see §9)
npm run temporal:live-check # ops KYC review → Tier 2 through a real Temporal server
npm run conductor:live-check # same through a real Conductor server
npm run money:live-check    # exactly-once transfer through a real Temporal money wor

# Validate – agent/MCP journeys as a real client (the hybrid layer)
#   invoke the `e2e-validator` skill (scope: through-phase-8 | full) – see docs/E2E-VA
#   it drives SmartChat + external-agent (OID4VP+MCP) and asserts token TTL / MFA / sc

# Other services
cd ../fraud-engine && npm test          # fraud-engine suite (27 tests)
```

Stop everything: Ctrl-C each `npm run dev` ; for orchestration containers `docker rm -f argus-temporal argus-conductor` .

A clean validation pass (copy/paste):

```
cd backend && npm install && npm run typecheck && npm test && \  
cd ../fraud-engine && npm install && npm run typecheck && npm test && \  
cd ../frontend && npm install && npm run typecheck
```

8 6. Optional: durable orchestration (Temporal + Conductor)

```
cd backend  
docker compose -f docker-compose.temporal.yml up -d    # Temporal (UI :8233)  
docker compose -f docker-compose.conductor.yml up -d   # Conductor (UI :5001 - wait ~  
npm run money:worker          # money transfers durable (TEMPORAL_MONEY_ENABLED=true)  
npm run conductor:worker      # agent ops via Conductor (CONDUCTOR_ENABLED=true) [or
```

With these on, J3/J6 transfers and the §3 agent-ops flows run as durable workflows you can watch in the Temporal/Conductor UIs. Both **degrade to in-process** if the server is unavailable — the runner never fails closed. Tear down: `docker rm -f argus-temporal argus-conductor` .

9 7. Journey × channel coverage matrix

Journey	Web (:5173)	SmartChat (:5173 /agent)	External agent (:5174)	Admin (:5173 /admin)	API/MCP
Auth / onboarding	✓ J1/J2	—	link step	review J13	✓
Transfer (money)	✓ J3	✓ J6	✓ J12	—	✓
On/off-ramp (cash in/out)	✓ J14/J15	—	—	—	✓
Earn / Lending	✓ J16/J17	—	—	accrue/liquidate	✓
P2P / FX / Cross-border	✓ J18/J19/J20	—	—	—	✓
Self-custody / Drops	✓ J21/J22	—	—	—	✓
Login-less VC checkout	✓ J23	—	device VC	—	✓
Invest / Collect	✓ J4/J5	—	—	listing lifecycle	✓
Trading	✓ J7	—	—	—	✓
Escrow	✓ J8	—	—	mediation J13	✓
Argus Pay	via API J9	—	pay_merchant tool	—	✓
On-chain wallet	✓ J10	—	—	reconciliation	✓
Credentials/VC	✓ J11	—	consumes VC	revoke	✓
Agent ops (Phase 15)	—	—	—	✓ §3	✓
Fraud	(triggered by money journeys)	✓	✓	freeze/cases	✓ §4

10 8. Where to add agents, skills & MCP services (expansion review)

The runner + scoped-skill + MCP scaffolding makes new capabilities cheap. High-value additions:

New operations skills (`backend/src/operations/skills/` , register in `skills/index.ts`)
— read/recommend/draft only:

- **Collections / dunning** — drafts outreach for negative balances / failed pays → human send (reuse the support pattern).
- **Dispute/chargeback analyst** — gathers escrow + payment context, recommends a mediation outcome → `compliance` gate (feeds J8/J13).
- **Transaction-monitoring triage** — turns fraud-engine cases into structured dispositions (clear / RFI / freeze / SAR) → routes to `compliance-filing` . (Closes the \$6 design item that's currently only sanctions+filing.)
- **Marketplace re-screening** — periodic issuer DD refresh feeding the Phase 8 listing lifecycle.
- **Reconciliation analyst** — drafts an incident from `reconciliation_findings` drift → SRE/compliance.

New MCP tools for external agents (`backend/src/routes/mcp.ts` , gated by scope $\cap$ client $\cap$ grant):

- `quote_asset` / `subscribe_asset` (read + escrow-gated invest) under a `marketplace:*` scope.
- `escrow_status` / `open_dispute` under `escrow:read` / `escrow:dispute` .
- `statement_export` under `statement:read` (already a tier op) — a common agent ask.
- A read-only `pay_status` companion to the existing `pay_merchant` .

New MCP servers (beyond the in-app tools):

- Wrap each **operations skill** as an internal MCP server so the same scoped toolset is reusable by Conductor/Temporal workers and external compliance tooling (the design's "internal skill servers").
- A **fraud-analytics MCP** exposing read-only `get_case` / `get_decision` / `query_blockchain_analytics` to analyst agents (maps to the \$6 Fraud & AML skill).

SmartChat intents (`smartchatService`): add `escrow.hold` , `pay.merchant` , `invest.quote` as classified intents → operation-token → existing services (mirrors

```
transfer.send ).
```

Orchestration: add a money saga workflow (ledger debit → on-chain settle → reconcile) on the Temporal money queue — the `src/money/` seam already supports multi-activity workflows.

Each addition should keep the invariants: **money/state via deterministic idempotent services; agents only recommend/draft; humans gate material actions; every tool scoped + audited.**

11 9. Troubleshooting

- Can't log in with a password → set `ALLOW_PASSWORD_AUTH=true` and restart; or use a passkey.
- `ACCOUNT_LOCKED` → `npm run reset:auth`.
- A product page returns 503 (`*_DISABLED`) → set the matching `*_ENABLED` flag:
`TRADING_ENABLED` / `ARGUS_PAY_ENABLED` / `BANK_RAILS_ENABLED` / `CARDS_ENABLED` / `BILLPAY_ENABLED` , and for the J14-J23 cluster `ONRAMP_ENABLED` / `OFFRAMP_ENABLED` / `TREASURY_ENABLED` / `LENDING_ENABLED` / `FX_ENABLED` / `FX_SETTLEMENT_ENABLED` / `CREATOR_DROPS_ENABLED` / `CHECKOUT_VP_ENABLED` .
- Borrow (`/borrow`) shows "no collateral" → buy Treasury (ATB) on `/earn` first (needs `TREASURY_ENABLED`); the loan pledges that holding.
- Fraud engine "unreachable, degrading open" → engine not running or `FRAUD_ENGINE_URL` /key mismatch.
- Temporal/Conductor "falling back to in-process" → server not up; start the compose file (§6).
- CORS errors from :5174 → ensure `CORS_ORIGIN` allows the agent app origin.
- On-chain journeys skipped → no Hedera creds; set `HEDERA_*` or accept simulated mode.